1. Names of the Two GenAI Programs

    a. ChatGPT (GPT-6)

    b. Google Gemini 3.5 Flash-Lite

2. How each GenAI was accessed

    a. ChatGPT was accessed in the browser at chatgpt.com

    b. Gemini was accessed in the browser at gemini.google.com

3. Prompt used

    a. The same prompt was given to both GenAIs:

```
Use two GenAI's to generate C++ code using objects, not functions, to implement the following
requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They
are essentially the same GenAI.):

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here (image credit: Wikimedia Commons):
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the
following constraints:
  - Each digit from 1 to 9 must appear exactly once in each row.
  - Each digit from 1 to 9 must appear exactly once in each column.
  - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least
17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as
many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells
that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is
to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at
least a) solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle,
consider the bottom right block, top right cell. Because of other numbers in the same row, column,
or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this
fashion, using these and more advanced deduction rules, it's possible to gradually fill in the
entire grid.
- That is not the approach we're going to use. Instead, we're going to implement a brute-force
search, in which we exhaustively try possibilities until we find a solution. This will be slower to
run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a
second rather than a few milliseconds.)
- Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We
identify an empty cell and deduce a list of candidates for what might go into that cell. If there
are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then
we must have made a mistake in filling in the k cells and return a no-solution-found result.
Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a
solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the
first candidate, then we try the next, and so on, until we either find a solution (k == 81) or
determine that no solution exists (all candidates report no solution, which again implies we took a
```

```
wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
  - The puzzle file name (e.g., puzzle1.txt)
  - The puzzle from the file printed out.
  - The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.
```

4. Generated Code

a. ChatGPT

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
using namespace std;

class SudokuSolver {
private:
    static const int SIZE = 9;
    int grid[SIZE][SIZE];
    int solutionCount;

    bool isSafe(int row, int col, int num) {
        for (int i = 0; i < SIZE; i++) {
            if (grid[row][i] == num || grid[i][col] == num) {
                return false;
            }
        }

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (grid[r][c] == num) {
                    return false;
                }
            }
        }

        return true;
    }
```

```
void printGrid() {
    for (int row = 0; row < SIZE; row++) {
        if (row % 3 == 0 && row != 0) {
            cout << "------+-------+------\n";
        }

        for (int col = 0; col < SIZE; col++) {
            if (col % 3 == 0 && col != 0) {
                cout << "| ";
            }

            if (grid[row][col] == 0) {
                cout << "_ ";
            } else {
                cout << grid[row][col] << " ";
            }
        }

        cout << '\n';
    }
}

bool hasValidInitialGrid() {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (grid[row][col] != 0) {
                int num = grid[row][col];
                grid[row][col] = 0;

                bool valid = isSafe(row, col, num);
                grid[row][col] = num;

                if (!valid) {
                    return false;
                }
            }
        }
    }

    return true;
}

void solveCell() {
    int row = -1;
    int col = -1;

    for (int r = 0; r < SIZE; r++) {
        for (int c = 0; c < SIZE; c++) {
            if (grid[r][c] == 0) {
                row = r;
                col = c;
                break;
            }
        }
```

```
            if (row != -1) {
                break;
            }
        }

        if (row == -1) {
            solutionCount++;
            cout << "\nSolution " << solutionCount << ":\n";
            printGrid();
            return;
        }

        for (int num = 1; num <= SIZE; num++) {
            if (isSafe(row, col, num)) {
                grid[row][col] = num;
                solveCell();
                grid[row][col] = 0;
            }
        }
    }

public:
    SudokuSolver() {
        solutionCount = 0;

        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                grid[row][col] = 0;
            }
        }
    }

    bool loadPuzzle(const string& filename) {
        ifstream input(filename);

        if (!input.is_open()) {
            cout << "Error: Could not open " << filename << ".\n";
            return false;
        }

        vector<char> cells;
        char ch;

        while (input >> ch) {
            if (ch >= '1' && ch <= '9') {
                cells.push_back(ch);
            } else if (ch == '_') {
                cells.push_back(ch);
            } else {
                cout << "Error: Invalid character in "
                     << filename << ".\n";
                return false;
            }
        }

        if (cells.size() != SIZE * SIZE) {
```

```
            cout << "Error: " << filename
                 << " must contain exactly 81 cells.\n";
            return false;
        }

        int index = 0;

        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                char value = cells[index++];

                if (value == '_') {
                    grid[row][col] = 0;
                } else {
                    grid[row][col] = value - '0';
                }
            }
        }

        return true;
    }

    void solve(const string& filename) {
        cout << "\n================================\n";
        cout << "Puzzle file: " << filename << '\n';
        cout << "================================\n";

        cout << "\nOriginal puzzle:\n";
        printGrid();

        solutionCount = 0;

        if (!hasValidInitialGrid()) {
            cout << "\nNo solution found\n";
            return;
        }

        solveCell();

        if (solutionCount == 0) {
            cout << "\nNo solution found\n";
        } else {
            cout << "\nTotal solutions found: "
                 << solutionCount << '\n';
        }
    }
};

int main() {
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";
        SudokuSolver solver;

        if (solver.loadPuzzle(filename)) {
            solver.solve(filename);
        }
```

```
    }

    return 0;
}
```

b. Gemini

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>

class SudokuBoard {
private:
    std::vector<std::vector<char>> grid;

public:
    SudokuBoard() : grid(9, std::vector<char>(9, '_')) {}

    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) return false;
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                file >> grid[i][j];
            }
        }
        return true;
    }

    void print() const {
        for (int i = 0; i < 9; ++i) {
            if (i > 0 && i % 3 == 0) {
                std::cout << "------+-------+------\n";
            }
            for (int j = 0; j < 9; ++j) {
                if (j > 0 && j % 3 == 0) {
                    std::cout << "| ";
                }
                std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    bool isValid(int row, int col, char val) const {
        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == val) return false;
            if (grid[i][col] == val) return false;
        }

        // Check 3x3 block constraints
        int startRow = row - row % 3;
        int startCol = col - col % 3;
```

```
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == val) return false;
            }
        }

        return true;
    }

    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; ++row) {
            for (col = 0; col < 9; ++col) {
                if (grid[row][col] == '_') {
                    return true;
                }
            }
        }
        return false;
    }

    char getCell(int row, int col) const {
        return grid[row][col];
    }

    void setCell(int row, int col, char val) {
        grid[row][col] = val;
    }
};

class SudokuSolver {
private:
    SudokuBoard board;
    std::string filename;
    int solutionCount;

    void solveRecursive() {
        int row, col;
        // Base case: If no empty cells remain, a solution is found
        if (!board.findEmptyCell(row, col)) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << " for " << filename << ":\n";
            board.print();
            std::cout << "\n";
            return; // Continue searching to find any additional solutions
        }

        // Try candidates '1' through '9'
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (board.isValid(row, col, digit)) {
                board.setCell(row, col, digit); // Make move
                solveRecursive();               // Recursive step
                board.setCell(row, col, '_');   // Backtrack
            }
        }
    }
```

```
public:
    SudokuSolver(const std::string& fname) : filename(fname), solutionCount(0) {}

    void solve() {
        std::cout << "=======================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "Original Puzzle:\n";

        if (!board.loadFromFile(filename)) {
            std::cout << "Error: Could not open or read file " << filename << "\n";
            std::cout << "=======================================\n\n";
            return;
        }

        board.print();
        std::cout << "\nSearching for solutions...\n\n";

        solutionCount = 0;
        solveRecursive();

        if (solutionCount == 0) {
            std::cout << "No solution found for " << filename << "\n";
        } else {
            std::cout << "Total solutions found: " << solutionCount << "\n";
        }
        std::cout << "=======================================\n\n";
    }
};

int main() {
    std::string puzzleFiles[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt",
"puzzle5.txt"};

    for (const auto& file : puzzleFiles) {
        SudokuSolver solver(file);
        solver.solve();
    }

    return 0;
}
```

5. Analysis

a. Correctness

Both programs compile fine and get the right answer for all five puzzles. Puzzle 1 matches solution1.txt, puzzle 4 has no solution and puzzle 5 has two, and both print every solution. The difference is bad input. ChatGPT checks that every character is 1-9 or an underscore, that there are exactly 81 cells, and that the starting digits don’t repeat. Gemini doesn’t check any of that. When I put an x in a puzzle file, Gemini treated it as a filled cell and printed a “solution” with the x still in it. ChatGPT has one small problem: if a file can’t be loaded it only prints an error, not the file name or “No solution found”.

b. Execution time

Both use the same search, so both make exactly 1541 recursive calls for the five puzzles. They both take the first blank cell and check 27 cells (row, column and block) for every digit they try. ChatGPT was still about 2.5 times faster, 0.83 ms against 2.05 ms on the cycle server. That's because ChatGPT stores the grid in a plain int array, while Gemini uses a vector of vectors, so every cell lookup goes through two vector index calls.

c. Space Complexity

Both programs keep one 9x9 grid and change it in place, so the only extra memory is the recursion stack, at most one call per blank cell. ChatGPT's grid is 81 ints, 324 bytes inside the object. Gemini's grid is 81 chars, but as a vector of 9 vectors it takes 10 separate heap allocations, about 321 bytes in total. ChatGPT also makes a temporary 81-char vector while it loads the file.

d. Maintainability

Gemini is organized better. It splits the program into a SudokuBoard class and a SudokuSolver class, uses std:: instead of using namespace std, and comments the main steps of the search. ChatGPT puts loading, checking, printing and solving all in one class with no comments, and hard codes the 3s for the block size. Neither one has the prologue comment header this course wants.

6. Selection and Justification

I picked ChatGPT's program as the base. Both get the right answers, but ChatGPT was about 2.5 times faster with the same number of recursive calls, and it already checks the puzzle file for bad input, which Gemini doesn't do at all. Gemini's one advantage was splitting the board and the solver into two classes. That is easy to bring over to ChatGPT by just moving methods into a new class, while fixing Gemini would mean adding all of the input checks and changing how it stores the grid. So building on ChatGPT and borrowing Gemini's class split gave me the good parts of both with the least rework.

7. Improvements made

a. Correctness

I kept all of ChatGPT's file checks and added one for a file with more than 81 cells. If a file can't be loaded, the program now still prints the file name and "No solution found". The search now builds the actual list of candidate digits for a cell and returns no-solution right away if that list is empty, which is how the assignment describes it. I also took Gemini's idea of putting the file name on every solution header.

b. Execution Time

The board now keeps a bitmask for each row, column and block of which digits are used, so checking a digit is one quick check instead of looking at 27 cells. The program also fills in the blank cell with the fewest candidates first instead of just the first blank cell. This cut the recursive calls from 1541 to 176 and the time from 0.83 ms to 0.40 ms.

c. Space Complexity

I changed the grid from ints to unsigned chars since a cell only holds 0 to 9. Even with the masks added, the board went from 324 bytes to 138 bytes. I also removed the temporary vector ChatGPT used while loading, so digits are read straight into the grid.

d. Maintainability

I split ChatGPT’s one class into a SudokuBoard class and a SudokuSolver class like Gemini did. I removed using namespace std and replaced the hard-coded 9s and 3s with named constants. I added the prologue comment block with program name, description, inputs, outputs, collaborators, other sources, author and creation date, and put comments on every line saying whether the code came from ChatGPT, Gemini or me.